



KIRA ROBOTICS

Sounds simple. The interesting part is that the path is not a simulated curve: the robot's position was captured by an external tracking system while it drove.

One recording, `data/short.json`, with this structure:

```

{
  // Recorded robot trajectory: waypoints in world coordinates [m].
  // Positions only – no orientation, no timestamps.
  "path": [
    [x0, y0],
    [x1, y1],
    ...,
    [xN, yN]
  ],

  // Robot footprint as a closed polygon, in the ROBOT frame.
  // The origin of this frame is base_link.
  "robot": [
    [x0, y0], [x1, y1], [x2, y2], [x3, y3]
  ],

  // The cleaning gadget: a line segment, same frame as the robot polygon.
  "cleaning_gadget": [
    [x0, y0], [x1, y1]
  ]
}

```

Note: the block above is annotated for readability. The actual file is plain JSON and contains no comments.

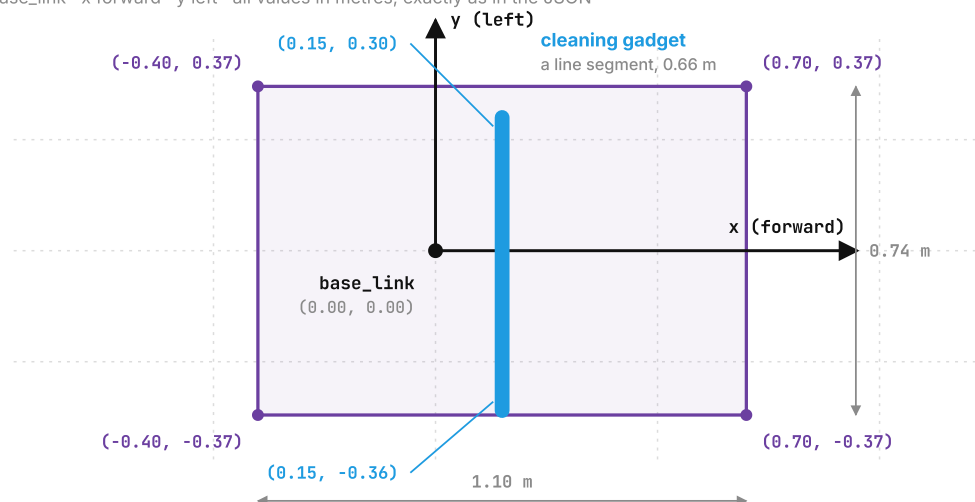
Take the input file as a parameter — `short.json` is one recording among others in this format.

Coordinate frames

`path` is in **world coordinates**. `robot` and `cleaning_gadget` are in the **robot frame**, whose origin is `base_link`, with **x pointing forward** and **y pointing left**.

The robot frame

origin = `base_link` · x forward · y left · all values in metres, exactly as in the JSON

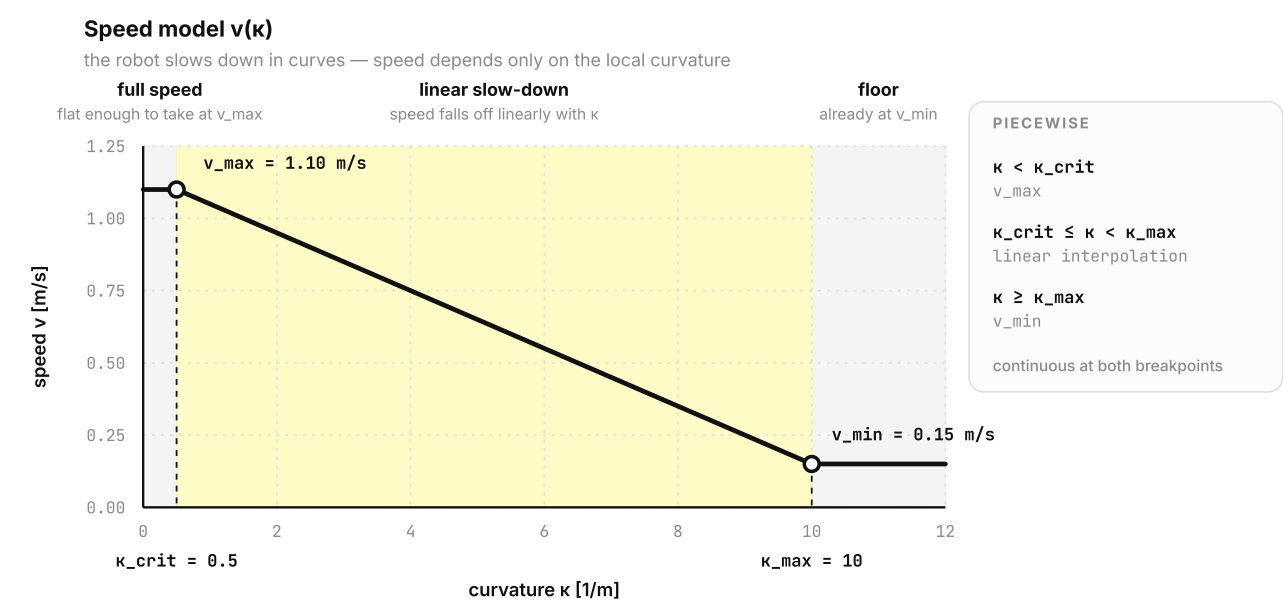


The path gives you **positions only** — the tracking system observed where the robot was, not which way it faced. Deriving a plausible heading from the path is part of the task, and it is what places the gadget in the world.

2. Speed model

The robot slows down in curves. Its speed depends only on the local path curvature κ [1/m]:

$$\begin{aligned} v(\kappa) &= v_{\max} && \text{for } \kappa < \kappa_{\text{crit}} \\ v(\kappa) &= v_{\max} - (v_{\max} - v_{\min})/(\kappa_{\max} - \kappa_{\text{crit}}) \cdot (\kappa - \kappa_{\text{crit}}) && \text{for } \kappa_{\text{crit}} \leq \kappa < \kappa_{\max} \\ v(\kappa) &= v_{\min} && \text{for } \kappa \geq \kappa_{\max} \end{aligned}$$



3. Task 1 — Computation (C++)

Write a modern, tested C++ program that reads such a JSON file and computes:

1. **Path length** in `[m]`.
2. **Cleaned area** in `[m2]` — the floor swept by the cleaning gadget as the robot moves along the path, whereby overlapping areas count only once.
3. **Traversal time** in `[s]` — how long the robot needs for the path under `v( $\kappa$ )`.

On the recorded data

The path was captured by an external tracking system while the robot drove, so the positions carry measurement uncertainty. It is a measurement, not an analytic curve — treat it as one.

Reasonable simplifications and approximations are expected, particularly for the curvature. We are not looking for the mathematically perfect answer; we are looking for a defensible one. Whatever you decide to do with the raw measurements, say so in your README and say why.

The same applies to the area: an exact swept region is a hard geometry problem, a good approximation is not. Pick a level of rigour you can justify and be explicit about the error it introduces.

4. Task 2 — Visualization

Using a graphical framework of your choice, produce meaningful visualizations of what you computed in Task 1.

"Meaningful" means someone can look at the output and see the result rather than read it off a number — for example the path in the plane, the region the gadget actually covered, where the robot had to slow down and why. Choose what you think communicates the result best; the choice itself is part of the answer.

You may do this in C++ or in a small companion script — just make it reproducible from the project.

5. Requirements

- The project must build with **CMake** and a reasonably modern **Clang / GCC / MSVC**. Building on **Linux is required**, Windows is optional.
- **Tests are mandatory** and must be executable via `ctest`. We want to see meaningful tests, not a placeholder that asserts `true`.
- `boost` **may be assumed available**. Every other dependency must either ship with the project or be handled by the build system, so that a clean checkout builds without manual setup steps.

- **Document the project in its own `README.md`** : how to build, how to run, what you computed, which approximations you chose and why, and what you would do with more time.
 - Share the project so that we can easily obtain the sources — a Git repository (GitHub / GitLab, public or invited) is ideal, a plain archive is fine.
-

6. Scope and ground rules

- **Budget roughly half a working day.** We would much rather see a small, clean, well-tested core than a large unfinished one. If you run out of time, say in your README what is missing and how you would finish it.
 - **AI assistants are explicitly allowed** — we use them ourselves. We will ask you about your code, so use them in a way that leaves you able to explain and defend every line.
 - If something in this document is ambiguous, make a decision, write it down, and move on. Documented assumptions are a good answer; guessing silently is not.
-

We will walk through your solution together in the next interview round. Bring the reasoning, not just the result.

Good luck — we are looking forward to it.